

Scalable Sentiment Classification: Word2Vec Vectorization and Document Length Benchmarking

Part 1: Text Data Exploration

This project was motivated by an interest in understanding how language can be represented computationally and used for machine learning tasks such as sentiment analysis. In a prior personal project, I implemented a Logistic Regression model to classify text sentiment and became interested in exploring distributed word representations such as Word2Vec. This assignment provided an opportunity to revisit and extend that work by integrating Word2Vec embeddings into a sentiment classification pipeline and evaluating their effectiveness.

The primary objective of this project was to construct a machine learning pipeline capable of classifying sentiment in text using Word2Vec for feature representation and Logistic Regression for classification. This approach allows semantic relationships between words to be captured through learned embeddings rather than relying solely on frequency-based representations.

Data and Data Processing

The dataset used in this project was the Sentiment140 dataset obtained from Kaggle. This dataset contains approximately 1.6 million tweets labeled according to sentiment polarity. The original dataset description indicates three possible sentiment categories: negative, neutral, and positive. However, exploratory analysis revealed that the available training data contained only two sentiment classes: negative (0) and positive (4). As a result, the classification task was treated as a binary sentiment classification problem.

Each record in the dataset contains multiple fields, including tweet text, user metadata, and sentiment labels. For the purposes of this analysis, only the sentiment label and tweet text were retained.

The tweet text required preprocessing to remove noise and ensure consistent input for the model. Regular expression (regex) techniques were used to remove URLs (http and https links), user mentions (@username), whitespace, punctuation and special characters. All text was converted to lowercase to ensure that words with different capitalization were treated consistently.

After cleaning, a working dataset was created containing only two columns: ["target"] the original sentiment label, and ["clean_text"] the preprocessed tweet text. Rows containing missing or empty text were removed to ensure valid input for downstream processing. The sentiment labels were then mapped from their original values (0 and 4) to binary labels (0 = negative, 1 = positive) to simplify classification.

The dataset was divided into training and testing subsets using scikit-learn's `train_test_split` function, with stratification applied to preserve class balance across both sets. This ensures that the model is evaluated on unseen data while maintaining consistent class distribution.

To generate numerical representations of text, the Word2Vec algorithm was applied to the corpus. Word2Vec learns dense vector representations of words based on their surrounding context. These embeddings capture semantic relationships between words and provide richer representations compared to traditional frequency-based methods.

Word2Vec Preprocessing

The preprocessing step for Word2Vec involved tokenizing each tweet into individual words and removing very short tokens that provided little semantic value. The resulting corpus consisted of over one million documents, approximately 13.5 million tokens, and a vocabulary of over 400,000 unique words.

The Word2Vec model was trained using the skip-gram architecture, which is well suited for learning representations of less frequent and contextually informative words. The embedding dimension was set to 200, providing sufficient representational capacity for the dataset size while maintaining computational efficiency. Because Logistic Regression requires a fixed-length feature vector per document, each tweet was converted into a document-level vector by averaging the embeddings of all words present in the tweet.

To generate numerical representations of text, the Word2Vec algorithm was applied to the corpus. Word2Vec learns dense vector representations of words based on their surrounding context. These embeddings capture semantic relationships between words and provide richer representations compared to traditional frequency-based methods.

The preprocessing step for Word2Vec involved tokenizing each tweet into individual words and removing very short tokens that provided little semantic value. The resulting corpus consisted of over one million documents, approximately 13.5 million tokens, and a vocabulary of over 400,000 unique words. The Word2Vec model was trained using the skip-gram architecture, which is well suited for learning representations of less frequent and contextually informative words. The embedding dimension was set to 200, providing sufficient representational capacity for the dataset size while maintaining computational efficiency.

Because Logistic Regression requires a fixed-length feature vector per document, each tweet was converted into a document-level vector by averaging the embeddings of all words present in the tweet.

The document vectors produced by Word2Vec were used as input features for a Logistic Regression classifier implemented using scikit-learn. The multinomial formulation was selected to allow for future expansion to multi-class sentiment classification, including a potential neutral category. Class weighting was also applied to mitigate any class imbalance introduced during preprocessing.

This model serves as a baseline for future experimentation with alternative embedding methods and parameter tuning, including adjustments to Word2Vec hyperparameters such as window size and training epochs.

Logistic Regression Results

	Class	Precision	Recall	F1
	0	0.76	0.75	0.75
	1	0.75	0.76	0.76
Accuracy				0.75
Macro Avg		0.75	0.75	0.75
Weighted Avg		0.75	0.75	0.75

The Logistic Regression model trained on Word2Vec document embeddings achieved an overall accuracy of 75% on the test set, with balanced precision, recall, and F1-scores across both sentiment classes. The negative and positive classes achieved F1-scores of 0.75 and 0.76 respectively, indicating that the model performed consistently across both categories without significant bias toward either class.

The results show that our embeddings successfully captured meaningful semantic information from the corpus, allowing the classifier to distinguish between positive and negative sentiment. Considering the source of the data being natural language found in an informal setting that includes abbreviations and context, the performance suggests a strong baseline for sentiment

classification. However, averaging word embeddings loses word order and contextual structure, which may lead to misclassifications when sentiment depends on specific word combinations or negation

Overall, these results confirm that Word2Vec provides an effective representation for sentiment classification and a baseline to explore different ways to improve it through hyperparameter tuning and alternative embedding methods such as GloVe.

Part 2: Sentiment Classification Using Word2Vec and Logistic Regression

The goal of this project was to determine whether Word2Vec embeddings could effectively capture sentiment in movie reviews and support accurate classification using a Logistic Regression model. Sentiment analysis is widely used in industry to understand public opinion from large volumes of text, such as product reviews, social media posts, and customer feedback. In the case of movie reviews, sentiment classification can help identify patterns in audience reception and better understand how viewers react to films.

Compared to my first analysis, this project focused more on evaluating the reliability of the modeling pipeline itself. In addition to building a sentiment classifier, I explored how Word2Vec training parameters affect performance and whether the model remains stable across different training configurations.

The Data

The dataset used for this project was the [IMDb Dataset of 50K Movie Reviews](#), obtained from Kaggle. It contains 50,000 labeled movie reviews, each classified as either positive or negative. The dataset includes two main variables: (1) review – the full text of a movie review and (2) sentiment – the sentiment label (positive or negative).

This dataset was chosen because it differs from the short-form text used in Analyze Me #1 while still focusing on sentiment classification. Movie reviews tend to be longer and contain richer descriptive language, which makes them particularly suitable for training word embeddings.

To prepare the data for modeling, the sentiment labels were converted into binary numeric form (negative = 0, positive = 1). A working dataframe was also created so the original dataset remained unchanged during preprocessing.

Data Cleaning and Preprocessing

The IMDb review dataset required several preprocessing steps to ensure the text was suitable for Word2Vec training. The raw reviews contained HTML artifacts, encoding issues, and inconsistent punctuation that could interfere with tokenization and embedding quality.

First, HTML tags such as `<br />` were removed after a punctuation check revealed frequent `<` and `>` characters, indicating embedded markup. HTML entities (e.g., `&`) were decoded to restore readable characters. All text was converted to lowercase to prevent duplicate tokens caused by capitalization differences.

Unicode normalization was applied to standardize punctuation and remove corrupted characters, ensuring that visually similar symbols were treated consistently. URLs and leftover control characters were also removed because they do not contribute meaningful sentiment information.

Most punctuation was removed while preserving apostrophes inside words so that contractions such as *don't* and *isn't* remained intact. Rows containing empty or missing cleaned text were dropped to ensure all documents contained usable content.

The cleaned text was then tokenized into individual words, and very short tokens were filtered out to reduce noise. Finally, the dataset was split into training and testing sets using an 80/20 stratified split to maintain class balance. Word2Vec embeddings were trained only on the training corpus to prevent data leakage and ensure a valid evaluation of model performance.

The Analysis

The main text representation used in this project was Word2Vec with the skip-gram architecture. Skip-gram was selected because it tends to perform well on datasets with larger vocabularies and is effective at learning semantic relationships between words. Key model parameters included: vector size: 200; minimum word frequency: 5; context window: 5; architecture: skip-gram; epochs tested: 5, 10, and 20.

Word2Vec parameters were adjusted after examining corpus statistics, including document count, vocabulary size, and average review length. Skip-gram (sg=1) was selected due to the relatively large vocabulary and descriptive nature of the reviews, while the context window was increased from 3 to 5 to better capture relationships across longer sentences. Multiple epoch settings were tested to determine whether additional training improved embedding quality and downstream classification performance.

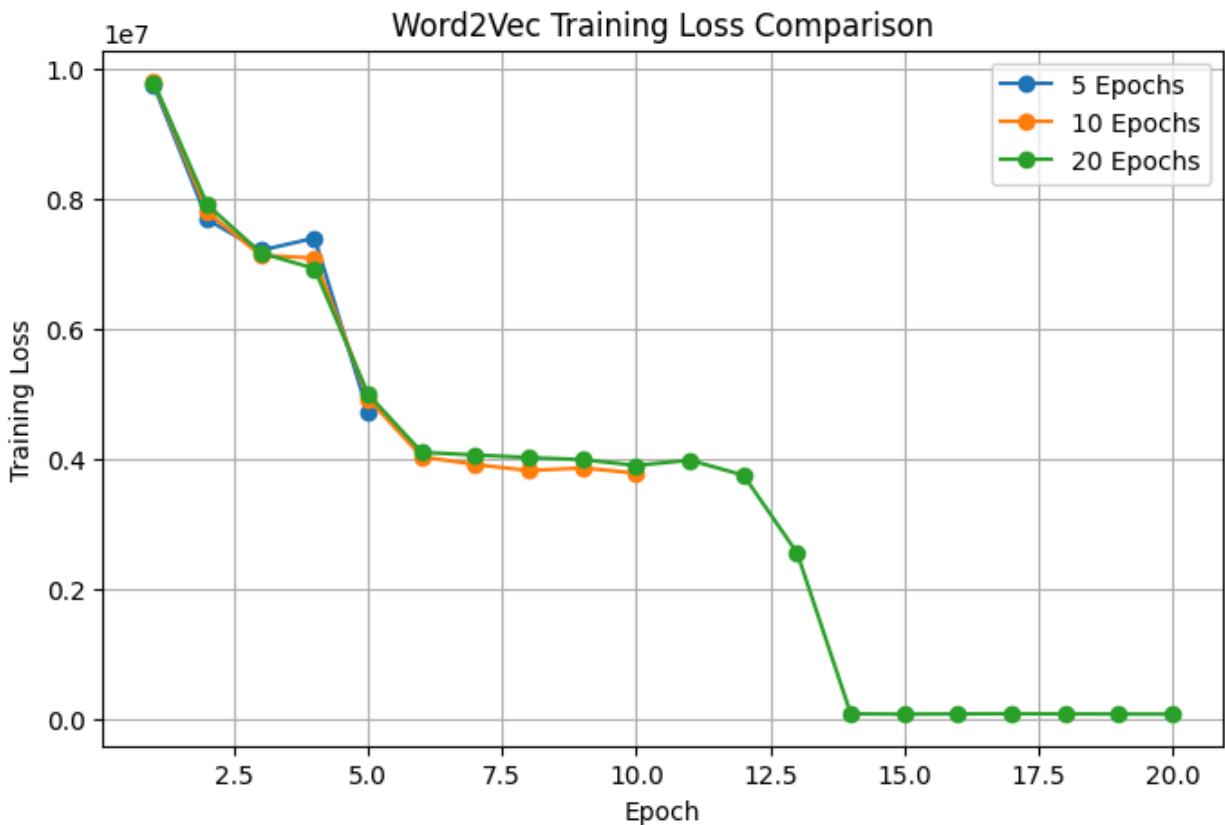
After training the Word2Vec model on the training corpus, each review was converted into a fixed-length document vector by averaging the word embeddings of the tokens contained in that review. These document vectors were then used as input features for a Logistic Regression classifier.

Corpus Characteristics

The training corpus contained approximately: 40,000 documents; 7.3 million total tokens; 108,977 unique words; average document length of about 184 tokens. These values indicate that the dataset contains a large amount of textual information and a relatively diverse vocabulary.

Epoch Comparison and Training Loss

To evaluate whether additional training improved the embeddings, Word2Vec models were trained using 5, 10, and 20 epochs. Training loss was tracked across epochs and visualized.



The results showed that the training loss decreased rapidly during the earliest epochs, indicating that the model learned the majority of useful semantic relationships quickly. Although the loss continued to decline slightly with additional epochs, classification performance remained identical across all three models (Accuracy ≈ 0.88 , F1 ≈ 0.88). Because the downstream classification results did not improve beyond 5 epochs, the 5-epoch model was selected as the most efficient configuration.